

Jurnal Modul 3: Synchronization & Deadlock

A. Persiapan Awal

1. Pastikan lingkungan praktikum

- Gunakan **Ubuntu** atau distro Linux lain yang sesuai.
- Pastikan **compiler** (gcc) atau **interpreter** (Python) tersedia untuk percobaan pemrograman.
- Jika menggunakan bahasa C, siapkan editor teks (mis. nano, vim) dan pastikan gcc terinstal (sudo apt-get install build-essential).

2. Pengetahuan Dasar

- Pahami struktur dasar pembuatan program C/Python (deklarasi, fungsi main, dll.).
- Ketahui konsep *thread* (bisa menggunakan *POSIX Threads* di C atau modul *threading* di Python).

B. Demo & Aktivitas Praktik

1. Observasi Race Condition Tanpa Sinkronisasi

- **Tujuan:** Melihat bagaimana *race condition* dapat terjadi pada data bersama (*shared data*) ketika beberapa *thread* mengaksesnya secara bersamaan tanpa proteksi.

Langkah (Contoh di C dengan Pthread):

1. Buat file `race.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define NUM_INCREMENTS 1000000

long counter = 0; // shared variable

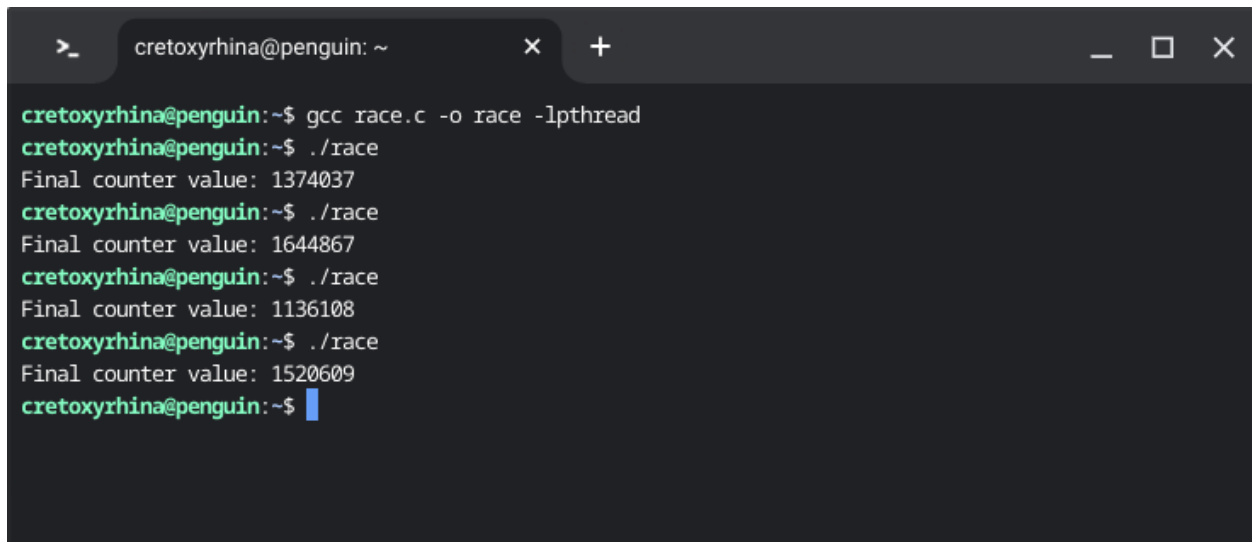
void* increment(void* arg) {
    for (long i = 0; i < NUM_INCREMENTS; i++) {
```

```
        counter = counter + 1;
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }
    // Tunggu semua thread selesai
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("Final counter value: %ld\n", counter);
    return 0;
}
```

2. Kompilasi dan jalankan:

```
gcc race.c -o race -lpthread
./race
```

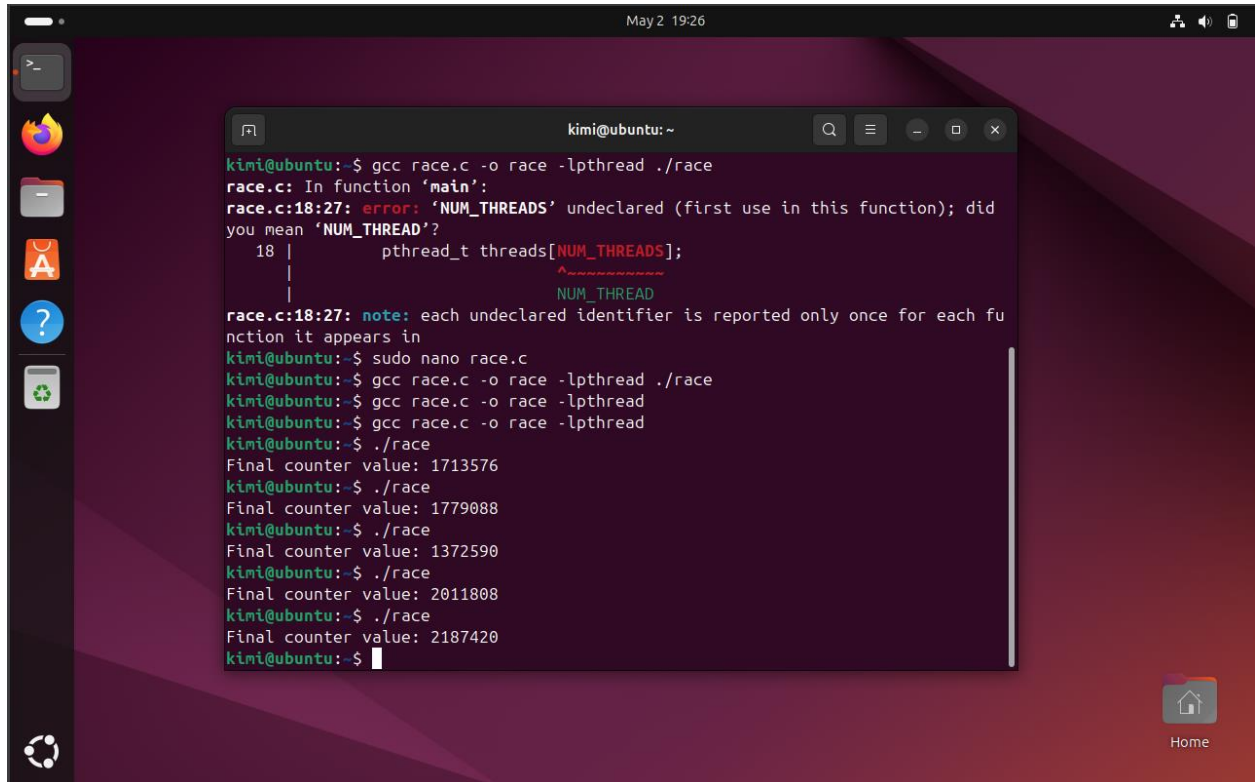


The screenshot shows a terminal window with the following content:

```
cretoxyrhina@penguin: ~
cretoxyrhina@penguin:~$ gcc race.c -o race -lpthread
cretoxyrhina@penguin:~$ ./race
Final counter value: 1374037
cretoxyrhina@penguin:~$ ./race
Final counter value: 1644867
cretoxyrhina@penguin:~$ ./race
Final counter value: 1136108
cretoxyrhina@penguin:~$ ./race
Final counter value: 1520609
cretoxyrhina@penguin:~$
```

3. Ulangi beberapa kali.

- a. Amati nilai akhir counter. Kemungkinan nilai final tidak konsisten (sering kurang dari $\text{NUM_THREADS} * \text{NUM_INCREMENTS}$), menunjukkan terjadinya *race condition*.
4. Diskusikan: Mengapa hasilnya tidak sama setiap kali dijalankan?



```
kimi@ubuntu: ~  
kimi@ubuntu:~$ gcc race.c -o race -lpthread ./race  
race.c: In function 'main':  
race.c:18:27: error: 'NUM_THREADS' undeclared (first use in this function); did  
you mean 'NUM_THREAD'?  
18 |         pthread_t threads[NUM_THREADS];  
    |                             ^  
    |                             NUM_THREAD  
race.c:18:27: note: each undeclared identifier is reported only once for each fu  
nction it appears in  
kimi@ubuntu:~$ sudo nano race.c  
kimi@ubuntu:~$ gcc race.c -o race -lpthread ./race  
kimi@ubuntu:~$ gcc race.c -o race -lpthread  
kimi@ubuntu:~$ gcc race.c -o race -lpthread  
kimi@ubuntu:~$ ./race  
Final counter value: 1713576  
kimi@ubuntu:~$ ./race  
Final counter value: 1779088  
kimi@ubuntu:~$ ./race  
Final counter value: 1372590  
kimi@ubuntu:~$ ./race  
Final counter value: 2011808  
kimi@ubuntu:~$ ./race  
Final counter value: 2187420  
kimi@ubuntu:~$
```

Mengalami race condition keadaan ini menyebabkan hasil output yang salah atau tidak sama

2. Menggunakan Mutex untuk Menghindari Race Condition

- **Tujuan:** Menunjukkan cara mengunci (*lock*) *shared resource* agar hanya satu *thread* yang dapat mengaksesnya pada satu waktu.

Langkah (Lanjutan C, race.c dimodifikasi):

1. Tambahkan library `pthread_mutex_t`:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>
```

```
#define NUM_THREADS 5
#define NUM_INCREMENTS 1000000

long counter = 0;
pthread_mutex_t lock; // Deklarasi mutex

void* increment(void* arg) {
    for (long i = 0; i < NUM_INCREMENTS; i++) {
        pthread_mutex_lock(&lock); // Critical section start
        counter++;
        pthread_mutex_unlock(&lock); // Critical section end
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    pthread_mutex_init(&lock, NULL); // Inisialisasi mutex

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter value with mutex: %ld\n", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

2. Kompilasi dan jalankan:

```
gcc race.c -o race -lpthread
./race
```

```
cretoxyrhina@penguin: ~  
cretoxyrhina@penguin:~$ gcc racemutex.c -o racemutex -lpthread  
cretoxyrhina@penguin:~$ ./racemutex  
Final counter value with mutex: 5000000  
cretoxyrhina@penguin:~$ ./racemutex  
Final counter value with mutex: 5000000  
cretoxyrhina@penguin:~$ ./racemutex  
Final counter value with mutex: 5000000  
cretoxyrhina@penguin:~$ ./racemutex  
Final counter value with mutex: 5000000  
cretoxyrhina@penguin:~$
```

3. Ulangi beberapa kali, amati nilai akhirnya.
 - a. Seharusnya selalu konsisten ($\text{NUM_THREADS} * \text{NUM_INCREMENTS}$).
4. Diskusikan: Bagaimana mekanisme *mutex* membantu mencegah *race condition*?

```
kimi@ubuntu: ~  
Final counter value: 1713576  
kimi@ubuntu:~$ ./race  
Final counter value: 1779088  
kimi@ubuntu:~$ ./race  
Final counter value: 1372590  
kimi@ubuntu:~$ ./race  
Final counter value: 2011808  
kimi@ubuntu:~$ ./race  
Final counter value: 2187420  
kimi@ubuntu:~$ sudo nano race.c  
kimi@ubuntu:~$ gcc race.c -o race -lpthread  
kimi@ubuntu:~$ ./race  
Final counter value: 5000000  
kimi@ubuntu:~$ sudo nano race.c  
kimi@ubuntu:~$ gcc race.c -o race -lpthread  
kimi@ubuntu:~$ ./race  
Final counter value with mutex: 5000000  
kimi@ubuntu:~$ ./race  
Final counter value with mutex: 5000000  
kimi@ubuntu:~$ ./race  
Final counter value with mutex: 5000000  
kimi@ubuntu:~$ ./race  
Final counter value with mutex: 5000000  
kimi@ubuntu:~$
```

Karena dibatasi akses nya ke resource dan hanya satu thread atau proses yang bisa mengaksesnya dalam satu waktu

3. Simulasi Deadlock Sederhana

- **Tujuan:** Memperlihatkan bagaimana dua *thread* dapat saling menunggu *resource* (mutex) yang dipegang oleh satu sama lain, menyebabkan *deadlock*.

Langkah (Contoh di C):

1. Buat file deadlock.c:

```
#include <stdio.h>  
#include <pthread.h>  
#include <unistd.h>  
  
pthread_mutex_t lockA;  
pthread_mutex_t lockB;
```

```

void* thread1(void* arg) {
    pthread_mutex_lock(&lockA);
    printf("Thread 1 locked A\n");
    sleep(1); // simulasi pemrosesan

    printf("Thread 1 trying to lock B...\n");
    pthread_mutex_lock(&lockB); // Menunggu B
    printf("Thread 1 locked B\n");

    // Lepaskan
    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);

    pthread_exit(NULL);
}

void* thread2(void* arg) {
    pthread_mutex_lock(&lockB);
    printf("Thread 2 locked B\n");
    sleep(1);

    printf("Thread 2 trying to lock A...\n");
    pthread_mutex_lock(&lockA); // Menunggu A
    printf("Thread 2 locked A\n");

    pthread_mutex_unlock(&lockA);
    pthread_mutex_unlock(&lockB);

    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&lockA, NULL);
    pthread_mutex_init(&lockB, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

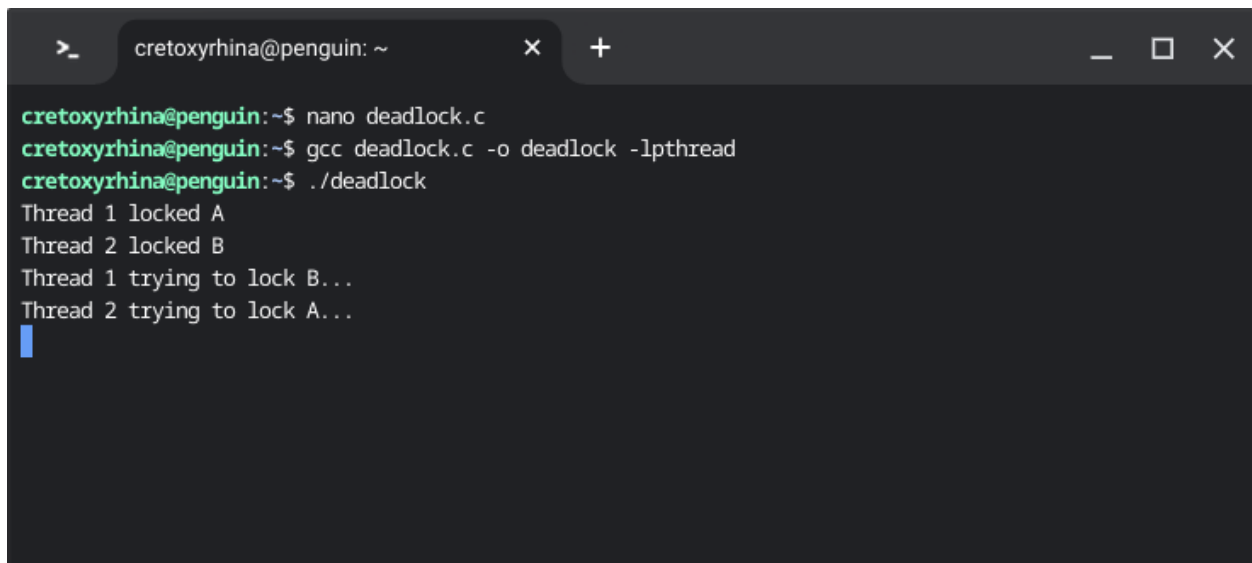
    pthread_mutex_destroy(&lockA);
    pthread_mutex_destroy(&lockB);
}

```

```
return 0;
}
```

2. Kompilasi dan jalankan:

```
gcc deadlock.c -o deadlock -lpthread
./deadlock
```



A screenshot of a terminal window with a dark background. The window title is 'cretoxyrhina@penguin: ~'. The terminal shows the following commands and output:

```
cretoxyrhina@penguin:~$ nano deadlock.c
cretoxyrhina@penguin:~$ gcc deadlock.c -o deadlock -lpthread
cretoxyrhina@penguin:~$ ./deadlock
Thread 1 locked A
Thread 2 locked B
Thread 1 trying to lock B...
Thread 2 trying to lock A...
```

The output shows two threads, Thread 1 and Thread 2, each holding a lock (A and B respectively) and then attempting to acquire the other's lock, leading to a deadlock state.

3. Amati output.

- a. Biasanya akan terjadi kebuntuan (kedua *thread* menunggu *mutex* yang sedang dipegang *thread* lain). Program akan “macet” dan tidak pernah menyelesaikan eksekusi.
4. Hentikan secara manual (Ctrl+C).
5. Diskusikan: Mengapa deadlock bisa terjadi? Apa empat syarat deadlock (mutual exclusion, hold and wait, no preemption, circular wait)?

```
kimi@ubuntu:~$ sudo nano deadlock.c
kimi@ubuntu:~$ gcc deadlock.c -o deadlock -lpthread
kimi@ubuntu:~$ ./deadlock
Thread 1 locked A
Thread 2 locked B
Thread 1 trying to lock B...
Thread 2 trying to lock A...
```

Deadlock terjadi saat dua atau lebih thread saling menunggu yang sedang di kunci oleh thread lain, jadi tidak ada yang bisa lanjut maka program macet. Dan empat syarat deadlock, 1 mutual exclusion: hanya bisa dipakai satu thread, 2 hold and

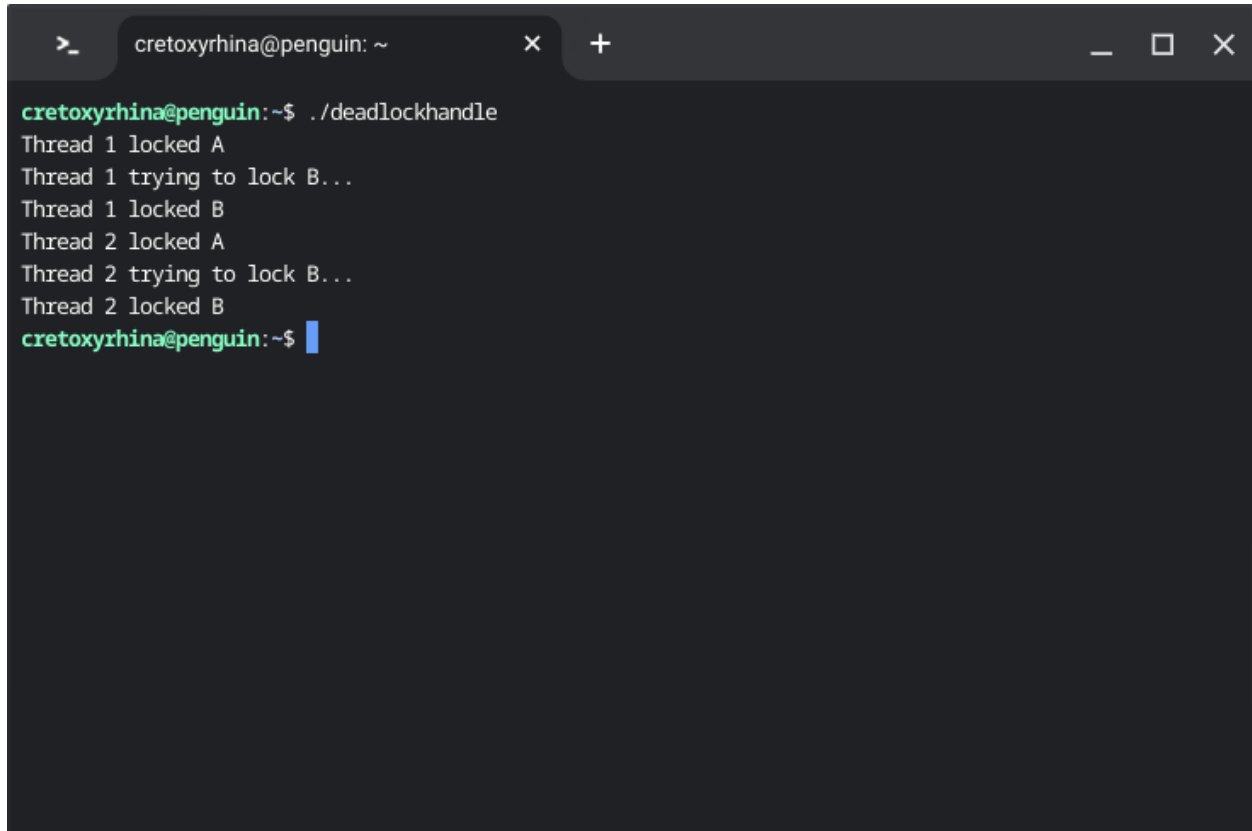
wait: thread memegang satu resource sambil menunggu yang lain, 3 no preemption: resource tidak bisa di rebut paksa, 4 circular wait: terjadi siklus saling tunggu antar thread

4. (Optional) Menunjukkan Penanganan Deadlock

- **Tujuan:** Mencontohkan bagaimana menghindari *deadlock* dengan mengunci mutex dalam urutan yang sama.

Ide Singkat:

- Jika kedua *thread* mengunci lockA lalu lockB dalam urutan sama, maka *deadlock* bisa dihindari.
- Anda dapat memodifikasi thread2 agar juga mengunci lockA dulu, baru lockB.

A terminal window titled 'cretoxyrhina@penguin: ~' with standard window controls. The terminal shows the execution of a program named 'deadlockhandle'. The output indicates that Thread 1 acquires lock A, then attempts to acquire lock B and succeeds. Subsequently, Thread 2 acquires lock A, attempts to acquire lock B, and also succeeds. The prompt returns to the user, indicating no deadlock occurred.

```
cretoxyrhina@penguin:~$ ./deadlockhandle
Thread 1 locked A
Thread 1 trying to lock B...
Thread 1 locked B
Thread 2 locked A
Thread 2 trying to lock B...
Thread 2 locked B
cretoxyrhina@penguin:~$
```

```

kimi@ubuntu:~$ sudo nano deadlock.c
kimi@ubuntu:~$ gcc deadlock.c -o deadlock -lpthread
kimi@ubuntu:~$ ./deadlock
Thread 1 locked A
Thread 2 locked B
Thread 1 trying to lock B...
Thread 2 trying to lock A...
^C
kimi@ubuntu:~$ sudo nano deadlock.c
kimi@ubuntu:~$ gcc deadlock.c -o deadlock -lpthread
kimi@ubuntu:~$ ./deadlock
Thread 1 locked A
Thread 1 trying to lock B...
Thread 1 locked B
Thread 2 locked A
Thread 2 trying to lock B...
Thread 2 locked B
kimi@ubuntu:~$

```

C. Tugas Mandiri

1. Penjelasan “Race Condition”

- Berikan definisi *race condition* dengan contoh nyata di luar konteks program (mis. dua orang menulis di papan tulis yang sama).
Dua orang yang mau menggunakan satu wc yang sama
- Tunjukkan hasil inkonsisten yang mungkin terjadi.
Yang akan terjadi wc tidak akan bisa digunakan secara bersamaan dan 2 orang akan stuck di tempat

2. Analisis Deadlock

- Berikan penjelasan singkat mengapa *circular wait* sangat menentukan deadlock.
Karena merupakan salah satu dari empat kondisi yang harus dipenuhi agar deadlock bisa terjadi
- Sertakan contoh lain (di dunia nyata atau simulasi) yang menggambarkan situasi deadlock.
Seperti koki membutuhkan bahan makanan tetapi pelayan lupa membeli bahan makanan